

Assignment Dossier: Operation Cyber-Histology

Post-Incident Pipeline Reconstruction and ML Engineering

RESTRICTED // FOR ML EXPERTS ONLY

To: Freelance Machine Learning Expert

From: Incident Response Team, BioHealth Diagnostics Global

Subject: Total Pipeline Failure Following Malicious Corporate Espionage

Urgency: CRITICAL / **WEIGHT:** 30 pts

Incident Report: Twenty-four hours ago, BioHealth Diagnostics was on the verge of deploying its next-generation multi-class clinical triage pipeline. Our core model registry, **AlexNet**, **VGG16**, and **ResNet18**, was calibrated to run seamlessly across our standardized diagnostic image profiles (`cells`, `chest`, `lesions` and `orgs`).

Before fleeing the campus to evade security, disgruntled former architect Dr. Viktor Vance executed an environment-wipe utility. The damage is extensive:

- All trained weights, production-ready source code, configuration registries, and evaluation automation scripts were permanently deleted.
- Forensics managed to scrape the local drive caches and recovered older, heavily unoptimized, and volatile draft copies of `data.py`, `models.py`, `train.py`, and `trainer.py`.
- All configuration assets (`config.json`), automated testing frameworks, and multi-dataset comparison tools were completely unrecoverable.

Preliminary execution attempts on the recovered code show that it is highly unstable. It suffers from a mix of immediate runtime exceptions, silent computational graph corruptions, numerical gradient explosions, and algorithmic anti-patterns.

Requested Action: Audit the volatile remains of the codebase, neutralize every mathematical and syntactic landmine left behind in the scripts, forge the missing configuration infrastructure from scratch, and deploy an automated multi-dataset benchmark runner to restore project integrity before the system completely goes dark. The incident response team expects the below listed deliverables as commits to a dedicated branch named `en_xxxx`, where `xxxx` is your student enrollment number, in the project repository https://git.fiw.fhws.de/teaching/mai/idl26_finalassignment, with clear commit messages indicating the nature of the changes made. **Deadline: 9.7.2026, midnight German time.**

1. **Complete Corrected Codebase:** A fully functional, production-grade version of the entire repository, including `data.py`, `models.py`, `train.py`, and `trainer.py`, alongside your newly engineered configuration and testing frameworks. The codebase must be modular, error-free, compatible with all four target datasets, and capable of executing training and predictions for all three models via native Python calls driven by your configuration structure.
2. **Incident Audit Log:** A technical table (submitted as `AUDIT.LOG.md`) itemizing every bug, corruption, or anti-pattern discovered within the recovered code. For each entry, you must explicitly document: (i) the file name, (ii) how the problem manifests, (iii) the mathematical or logical root cause of the failure, (iv) the structural correction implemented, and (v) the exact git commit hash containing the fix.
3. **Consolidated Benchmark Report:** A brief evaluation report (submitted as `REPORT.md` or `REPORT.pdf` in your repository root) comparing the final performance of the corrected models across all configuration permutations. This must include a clean summary table capturing key metrics (accuracy, precision, recall, and macro F1-score) along with architectural recommendations for optimal dataset/model pairings based on your observed

benchmarks. Expected **minimal accuracy** performance on test data: cells: 90%, chest: 87%, lesions: 67%, orgs: 83%.

System State and Engineering Requirements

The forensic team has been able to observe some patterns in the types of failures that are occurring when attempting to run the recovered code. So far, they have identified the following categories of issues that are plaguing the system and could be a useful starting point for your audit and reconstruction efforts:

Crashing code / runtime errors: Bugs that cause Python or PyTorch to throw an error and halt execution, e.g., syntax typos, shape mismatches, and incorrect data types.

Silent logical flaws: Hidden logical flaws where the scripts run smoothly without crashing, but the underlying math or logic is completely invalid, e.g., broken data boundaries, hidden data leaks, or shadowed variables that silently break the pipeline.

Numerical and gradient failures: Situations where models flatline or explode into useless numbers. You need to find out why the mathematical signals inside the network are getting trapped, neutralized, or blown out of proportion during execution.

Rigid infrastructure: Rigid, inflexible parts of the code that must be cleaned up into dynamic, modular variables so that the entire training and testing pipeline can be controlled cleanly through a single external configuration file.

OPEN // FOR ALL ML EXPERTS

To: Freelance Machine Learning Expert

From: Executive Board, BioHealth Diagnostics Global

Subject: Environmental Impact - Green Initiative

Urgency: MEDIUM / WEIGHT: 15 pts

Issue: The executive board recognizes the critical importance of sustainability and environmental responsibility in our operations. In light of the recent incident and the subsequent need to rebuild our machine learning pipelines, we are committed to ensuring that our recovery efforts align with our green initiative goals. We expect that all code optimizations, model training processes, and computational resource management strategies implemented during the recovery will prioritize energy efficiency and minimize carbon footprint.

Requested Action: You are required to refactor the restored models to propose a network architecture that drastically reduces computational time and energy footprint while preserving baseline classification accuracy. Heavy, bloated topologies are no longer acceptable for our diagnostic devices.

The board expects you to implement aggressive efficiency strategies directly into your model definitions as additional commits in your development branch. **Deadline: 9.7.2026, midnight German time.**

Your specific optimization targets are:

1. **Architectural Downscaling:** Come up with an architecture with reduced complexity to minimize runtime and memory usage while maintaining performance.
2. **Efficiency Verification Matrix:** Expand your automated orchestration runner to profile and log the resource consumption of each model. For every dataset-model permutation, your tracking metrics must now include: (i) total training runtime duration, (ii) inference

latency per sample, and (iii) peak memory consumption during training and inference phases.

3. **Green Initiative Analysis:** Include a dedicated section in your final report (`REPORT.md`) evaluating the trade-offs between model complexity and performance. You must quantitatively prove that your streamlined, green-optimized model achieves comparable accuracy to the original configurations while operating at a fraction of the computational cost.

OPEN // FOR ALL ML EXPERTS

To: Freelance Machine Learning Expert

From: Executive Board, BioHealth Diagnostics Global

Subject: New Dataset - organs

Urgency: HIGH / WEIGHT: 25 pts

Issue: New study has yielded new data - **organs**. Though the dataset is so far small, there is an accute need to bring these data to use. Simple integration into existing pipline is unlikely to deliver sufficient accuracy. Innovative solutions are required to leverage the new data and extract maximum value from it.

Requested Action: Specialized strategy to successfully integrate this low-sample dataset into the restored pipeline is needed.

The board expects you to implement a robust data-leveraging framework. These specific model developments shall be additional commits in your development branch. **Deadline: 9.7.2026, midnight German time.**

Your specific architectural requirements are:

1. **Knowledge Transfer Adaptation:** Adapt your pipeline to utilize a data-efficient training regime. Determine how to exploit structural features learned from your larger, existing image profiles to maximize performance on the new, limited organs data.
2. **Scarce-Data Benchmark Matrix:** Expand your automated runner to track and log the performance behavior of this new data under different training states (e.g., training from completely scratch vs. leveraging transferred feature knowledge).
3. **Data-Scarcity Post-Mortem:** Document a dedicated analysis in your final report (`REPORT.md`). Show quantitatively the impact of your innovative approach compared to standard baseline training. The board hopes for at least 50% accuracy on the test set.